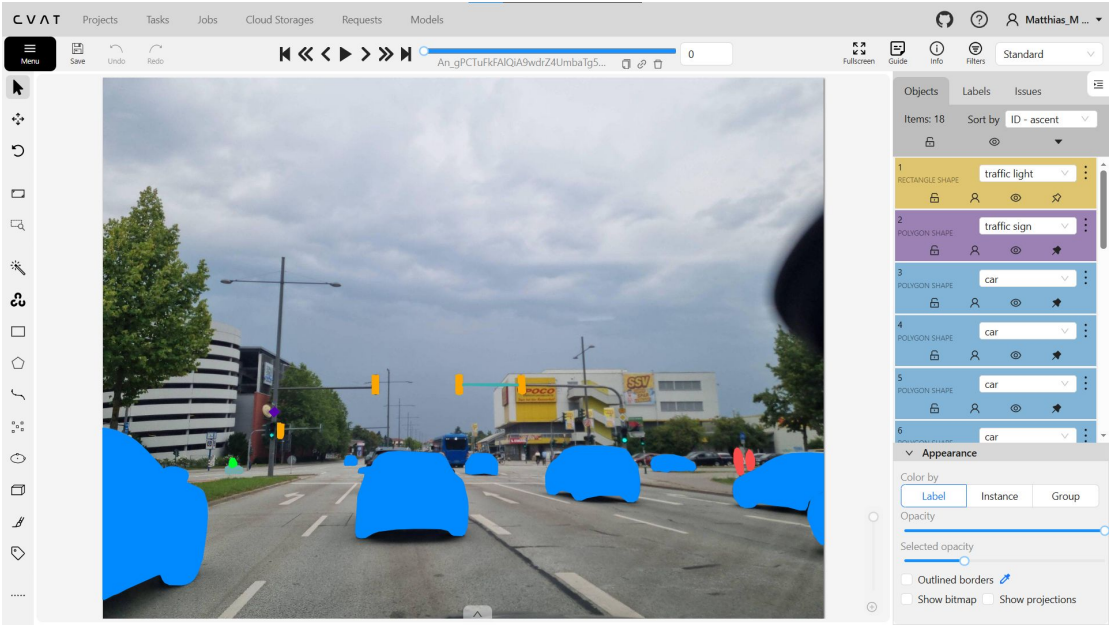
# Projektbesprechung 15.12

Gruppe P1b

# Annotation Testdaten Regensburg




- Bildextraktion durch Mapillary API
  → Kreuzungen in Regensburg
- Zuschneiden und bearbeiten der Bilder nötig
- Annotation durch CVAT
  → Cloud-Version (kostenfrei)
- Annotation in 17 Klassen
  → identisch zu Trainingsdaten
- Zeitaufwand ca. 2h pro Bild
  → 1. Ziel: 5 Bilder

# Training Classes

```
road: 0
sidewalk: 1
rail track: 2
building: 3
guard rail: 4
pole: 5
traffic light: 6
traffic sign: 7
terrain: 8
sky: 9
person: 10
rider: 11
car: 12
truck: 13
bus: 14
train: 15
motorcycle: 16
bicycle: 17
```

```
labels = [
    #       name                     id    trainId   category            catId     hasInstances   ignoreInEval   color
    Label(  'unlabeled'            ,  0 ,      255 , 'void'            , 0       , False        , True         , (  0,  0,  0) ),
    Label(  'ego vehicle'          ,  1 ,      255 , 'void'            , 0       , False        , True         , (  0,  0,  0) ),
    Label(  'rectification border' ,  2 ,      255 , 'void'            , 0       , False        , True         , (  0,  0,  0) ),
    Label(  'out of roi'           ,  3 ,      255 , 'void'            , 0       , False        , True         , (  0,  0,  0) ),
    Label(  'static'               ,  4 ,      255 , 'void'            , 0       , False        , True         , (  0,  0,  0) ),
    Label(  'dynamic'              ,  5 ,      255 , 'void'            , 0       , False        , True         , (111, 74,  0) ),
    Label(  'ground'               ,  6 ,      255 , 'void'            , 0       , False        , True         , ( 81,  0, 81) ),
    Label(  'road'                 ,  7 ,        0 , 'flat'            , 1       , False        , False        , (128, 64,128) ),
    Label(  'sidewalk'             ,  8 ,        1 , 'flat'            , 1       , False        , False        , (244, 35,232) ),
    Label(  'parking'              ,  9 ,      255 , 'flat'            , 1       , False        , True         , (250,170,160) ),
    Label(  'rail track'           , 10 ,        2 , 'flat'            , 1       , False        , True         , (230,150,140) ),
    Label(  'building'             , 11 ,        3 , 'construction'    , 2       , False        , False        , ( 70, 70, 70) ),
    Label(  'wall'                 , 12 ,      255 , 'construction'    , 2       , False        , False        , (102,102,156) ),
    Label(  'fence'                , 13 ,      255 , 'construction'    , 2       , False        , False        , (190,153,153) ),
    Label(  'guard rail'           , 14 ,        4 , 'construction'    , 2       , False        , True         , (180,165,180) ),
    Label(  'bridge'               , 15 ,      255 , 'construction'    , 2       , False        , True         , (150,100,100) ),
    Label(  'tunnel'               , 16 ,      255 , 'construction'    , 2       , False        , True         , (150,120, 90) ),
    Label(  'pole'                 , 17 ,        5 , 'object'          , 3       , False        , False        , (153,153,153) ),
    Label(  'polegroup'            , 18 ,      255 , 'object'          , 3       , False        , True         , (153,153,153) ),
    Label(  'traffic light'        , 19 ,        6 , 'object'          , 3       , False        , False        , (250,170, 30) ),
    Label(  'traffic sign'         , 20 ,        7 , 'object'          , 3       , False        , False        , (220,220,  0) ),
    Label(  'vegetation'           , 21 ,      255 , 'nature'          , 4       , False        , False        , (107,142, 35) ),
    Label(  'terrain'              , 22 ,        8 , 'nature'          , 4       , False        , False        , (152,251,152) ),
    Label(  'sky'                  , 23 ,        9 , 'sky'             , 5       , False        , False        , ( 70,130,180) ),
    Label(  'person'               , 24 ,       10 , 'human'           , 6       , True         , False        , (220, 20, 60) ),
    Label(  'rider'                , 25 ,       11 , 'human'           , 6       , True         , False        , (255,  0,  0) ),
    Label(  'car'                  , 26 ,       12 , 'vehicle'         , 7       , True         , False        , (  0,  0,142) ),
    Label(  'truck'                , 27 ,       13 , 'vehicle'         , 7       , True         , False        , (  0,  0, 70) ),
    Label(  'bus'                  , 28 ,       14 , 'vehicle'         , 7       , True         , False        , (  0, 60,100) ),
    Label(  'caravan'              , 29 ,      255 , 'vehicle'         , 7       , True         , True         , (  0,  0, 90) ),
    Label(  'trailer'              , 30 ,      255 , 'vehicle'         , 7       , True         , True         , (  0,  0,110) ),
    Label(  'train'                , 31 ,       15 , 'vehicle'         , 7       , True         , False        , (  0, 80,100) ),
    Label(  'motorcycle'           , 32 ,       16 , 'vehicle'         , 7       , True         , False        , (  0,  0,230) ),
    Label(  'bicycle'              , 33 ,       17 , 'vehicle'         , 7       , True         , False        , (119, 11, 32) ),
    Label(  'license plate'        , -1 ,       -1 , 'vehicle'         , 7       , False        , True         , (  0,  0,142) ),
]
```

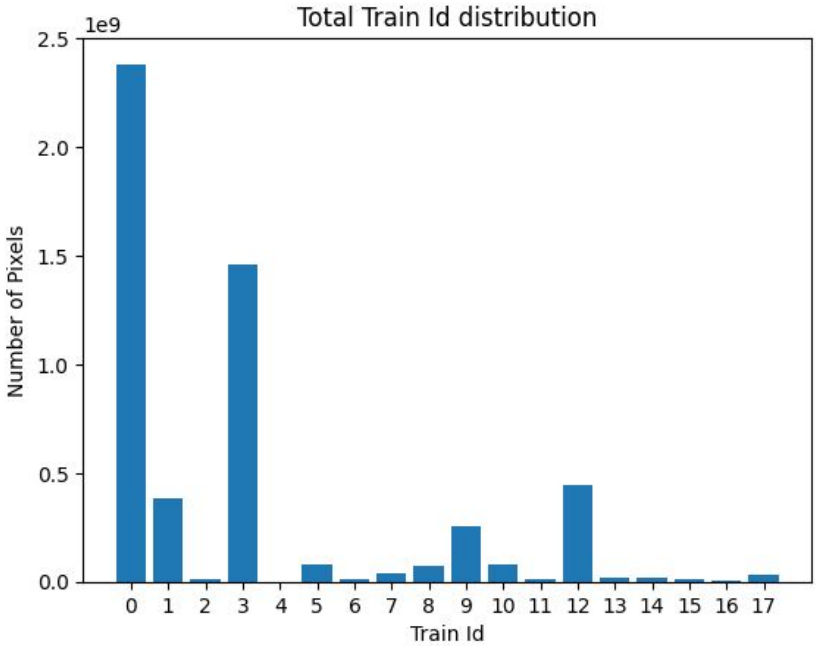
# Label Ids → Train Ids

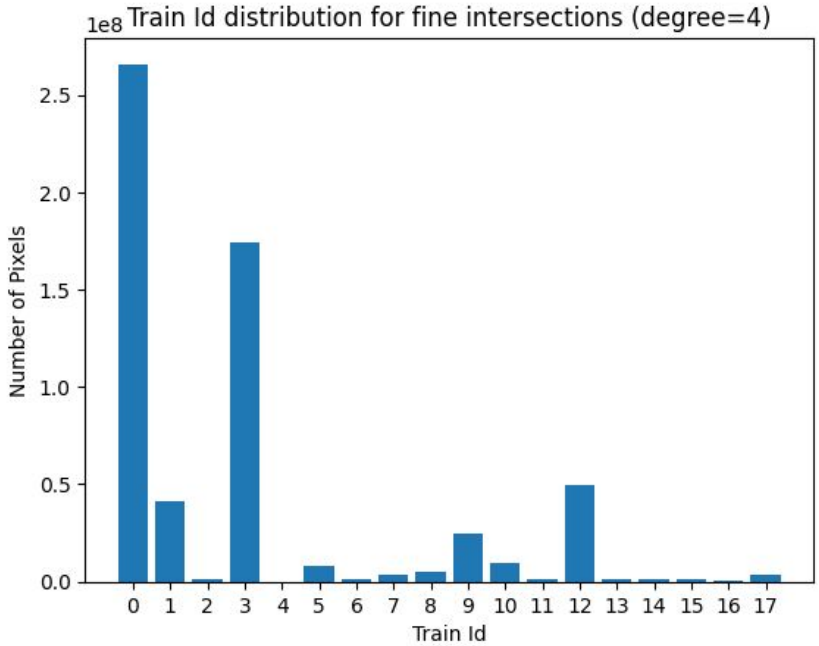
- Unwichtige Klassen auf ignore_index=255 gesetzt





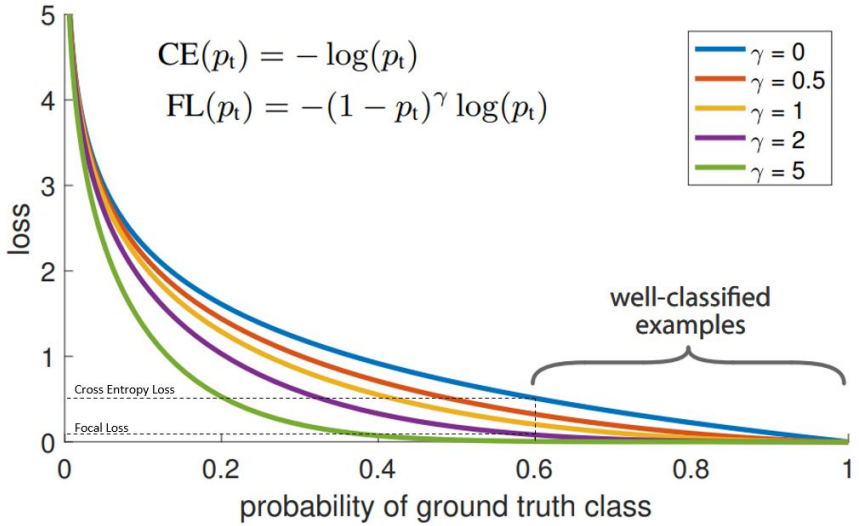
# Class Imbalance



# Focal Loss

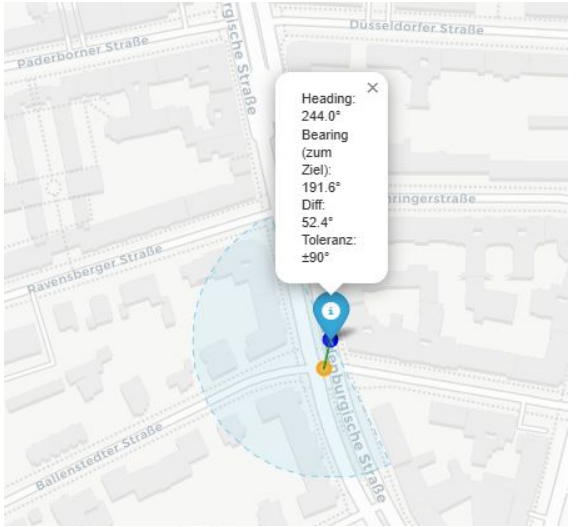
- Bei Class Imbalance wird CE Loss von leichten Beispielen dominiert (z.B. Road)
- Idee: Loss für leicht klassifizierte Beispiele reduzieren
- $\gamma$: Focal Exponent bestimmt wie stark der Loss reduziert wird

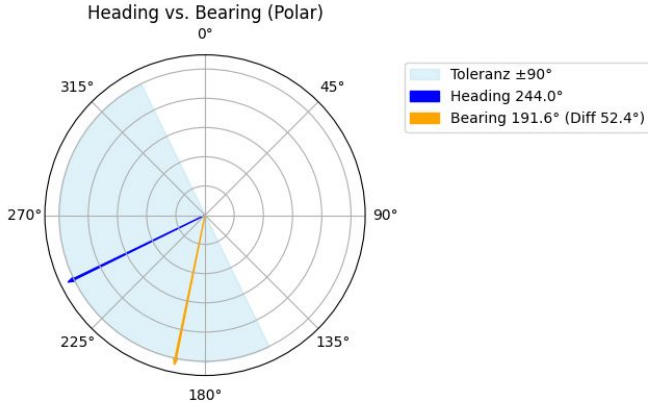


# Data Heading Filtering

Wie erwartet : ~50% der Bilder verbleiben:

```
Vergleich Input -> Output (vor/nach Heading-Filter):
coarse_4: 1912 -> 934
coarse_3: 6269 -> 3032
fine_4: 576 -> 300
fine_3: 1655 -> 845
```



# Model Comparison

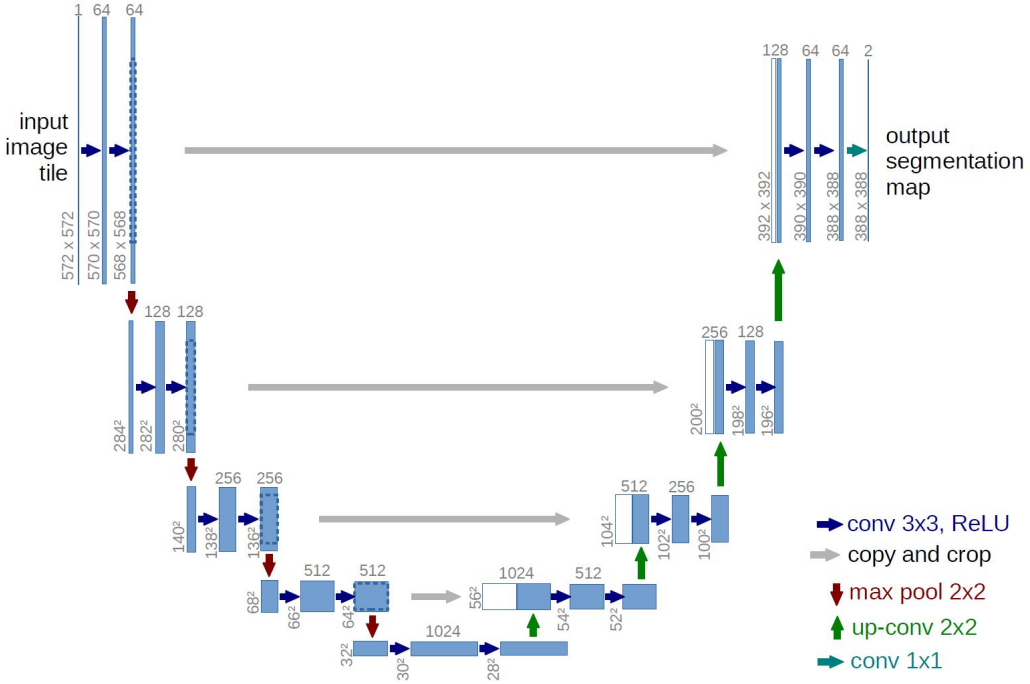
- U-Net gute Performance
- DeepLab State of the Art

| Architecture | Training time or Speed | Semantic segmentation | Instance Segmentation | Object Detection | Intersection over Union(IoU) |
|---|---|---|---|---|---|
| FCN | Fine tuning the coarse FCN-32s takes 3 days on a single GPU, and about one day to upgrade to FCN-16s and FCN-8s versions. FCN model takes less than one-fifth of a second for a typical image, showcasing the efficiency of their approach for semantic segmentation tasks. | Yes. FCNs can efficiently learn to make dense predictions for per-pixel tasks like semantic segmentation. | No, FCN cannot be used to perform instance segmentation | Yes. Object detection is done using CNN models to the task of object extraction from high spatial resolution imagery | IoU values for FCN model generally ranges from 0.35-0.55. |
| U-Net | Segmentation of a 512x512 image takes less than a second on a recent GPU | Yes, U-Net is designed for semantic segmentation | Yes, instance segmentation is possible using U-Net architecture | No, object detection is not supported by U-Net | IoU values for U-Net usually ranges from 0.50-0.70. |
| Deep Lab | The training time of Deep lab is specified as 8 FPS in NVidia Titan X GPU. It varies according to factors such as dataset size, model architecture, hardware used etc. | Yes, Deep lab is specifically designed for semantic image segmentation tasks | Yes, Deep lab is focused on instance segmentation tasks | No, Deep lab is not explicitly designed for object detection tasks | IoU values for Deep Lab usually ranges from 0.60-0.80. |
| SegNet | SegNet is designed to be significantly smaller and faster than other competing architectures, making it efficient for tasks such as road scene understanding. SegNet achieves high scores in various metrics compared to other models, indicating its efficiency in terms of training speed | Yes, SegNet architecture is specifically designed for semantic pixel-wise segmentation tasks | No, SegNet architecture, is focused on semantic pixel-wise segmentation rather than instance segmentation. | No, SegNet architecture is primarily designed for image segmentation tasks rather than object detection. | IoU values for SegNet usually ranges from 0.4-0.6. |

Mohan et al. - 2024 - A Comprehensive Review of Image Segmentation Architectures

# U-Net

- PyTorch Implementierung nac Ronneberger et al.
- Encoder-Decoder Architektur
- Skip Connections



# Segformer

- PyTorch Implementierung (nach Xie et al.)
- Während das U-Net auf CNNs basiert, basiert SegFormer auf Transformern

- **Patch-Embeddings:** Das Bild wird nicht Pixel-für-Pixel verarbeitet, sondern in überlappende „Patches“ zerlegt
- **Hierarchischer Encoder:** verringert die Bilder ebenfalls, speichert am Ende aber keinen Tensor, sondern Featuremaps, die dann beim decoden gleichzeitig vermischt werden

# Model Training with PyTorch

- CityscapesDataset Klasse für DataLoader erstellt
- Train-/Validation-Loops
- Loss: Focal Loss ($\gamma$=1, ignore_index=255)
- Optimizer: Adam
- Metriken:
  - Loss
  - Pixel Accuracy
  - mIoU
- Checkpoints (periodisch & best validation loss)
- Logging:
  - tqdm progress bar mit Metriken
  - TensorBoard (Logs mit SummaryWriter)

# Augmentierung

- Für jedes “native”- Bild wird eine Augmentierung erzeugt

  Diese wird Zufällig von:

- Spiegeln
- Kontrast/Helligkeits Manipulation
- Unschärfe
- Random Crop mit unterschiedlicher Größe

```python
# Resize auf feste Größe, dann EINE starke Veränderung wählen
train_transform = A.Compose([
    A.Resize(height=512, width=1024),

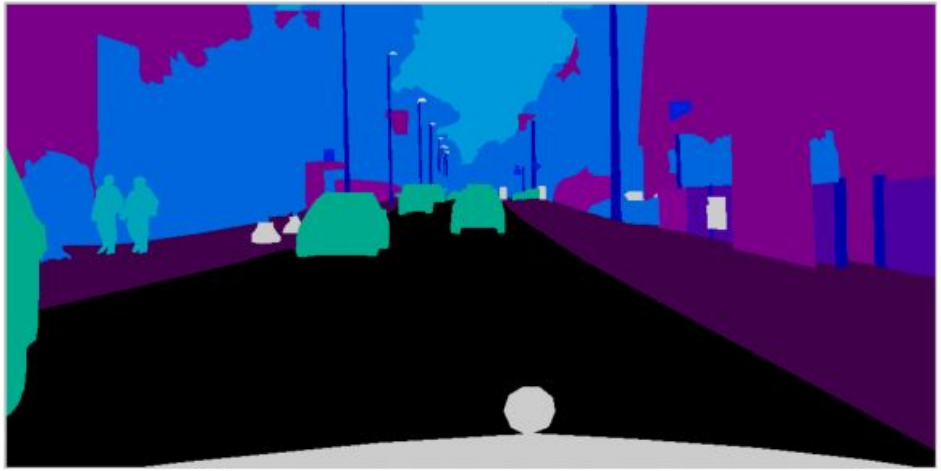
    # 2. "Eines von diesen Dingen tun" (Jede Option hat p=1.0 Chance gewählt zu werden)
    A.OneOf([
        A.HorizontalFlip(p=1.0),
        A.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1, p=1.0),
        A.GaussianBlur(p=1.0),
        A.RandomResizedCrop(
            size=(512,1024),
            scale=(0.7, 0.9),
            ratio=(2,2),
            p=1.0)
        ]),
 # https://discuss.pytorch.org/t/why-image-datasets-need-normalizing-with-means-and-stds-s
    A.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225)),
    ToTensorV2()
])
```

# Das Original Bild

Original (nur Resize) - Bild



Original (nur Resize) - Label

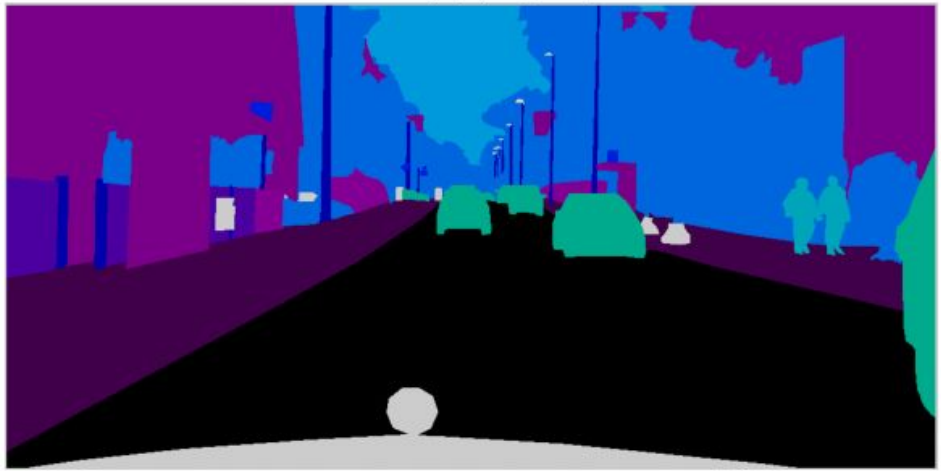


# Gespiegelt

Horizontal Flip (Spiegeln) - Bild



Horizontal Flip (Spiegeln) - Label

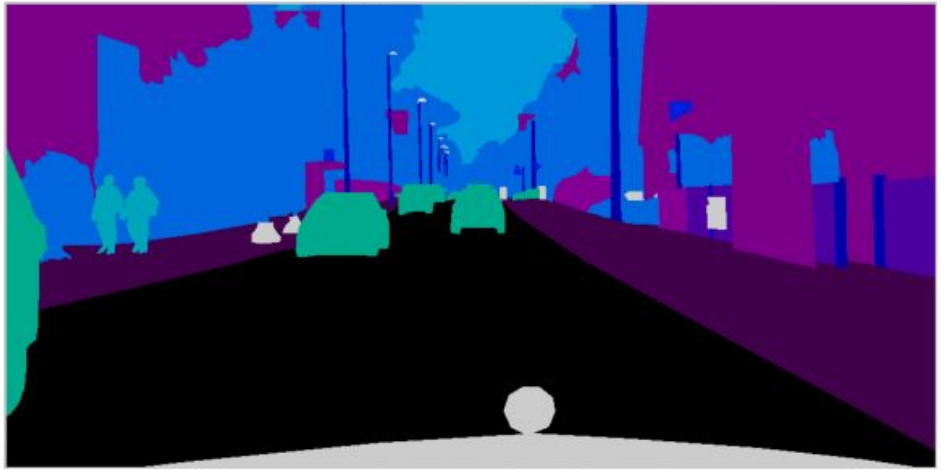


# Color Jitter

Color Jitter (Farbe) - Bild



Color Jitter (Farbe) - Label



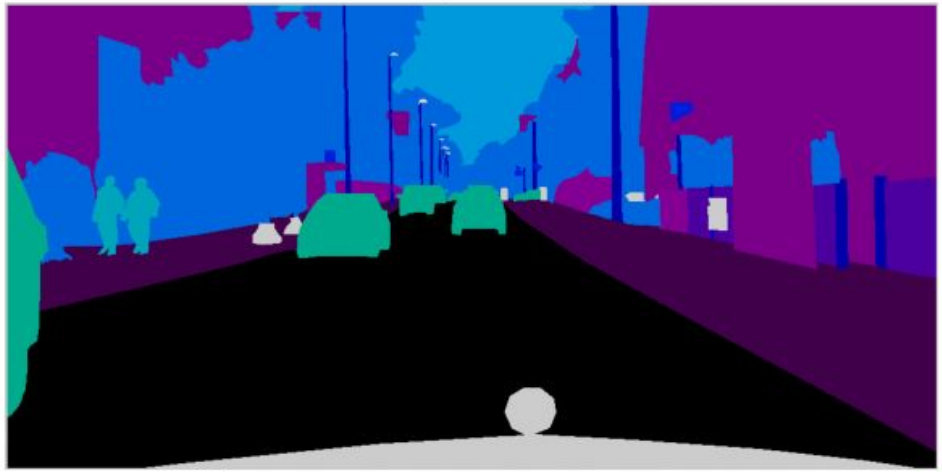
- Die **Helligkeit**, **Kontrast** und **Saturation** des Bildes wird zufällig um einen Faktor zwischen **0.8 - 1.2** angepasst.
- Der **Farbton** (die reine Farbe selbst, z. B. Verschiebung von Rot zu Orange oder Blau) wird zufällig um einen Wert zwischen **-0.1 und +0.1** verschoben

# Unschärfe

Gaussian Blur (Unschärfe) - Bild



Gaussian Blur (Unschärfe) - Label

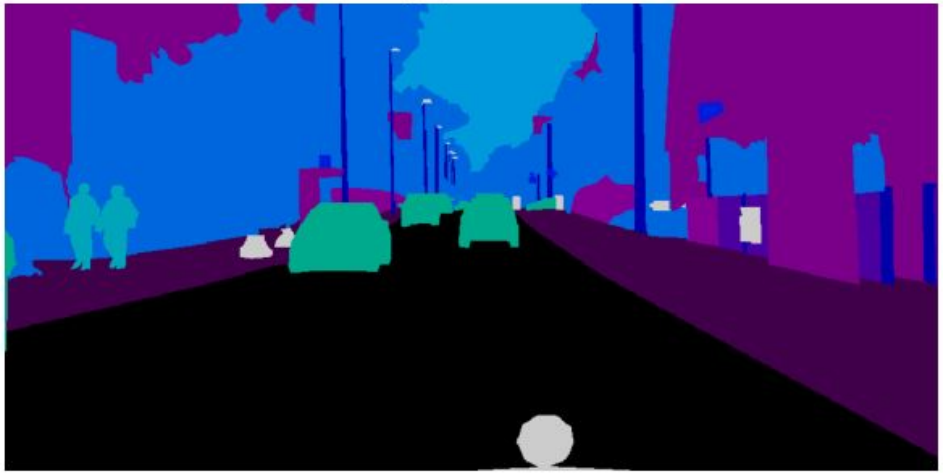


# Random Crop

Random Crop (Ausschnitt) - Bild



Random Crop (Ausschnitt) - Label

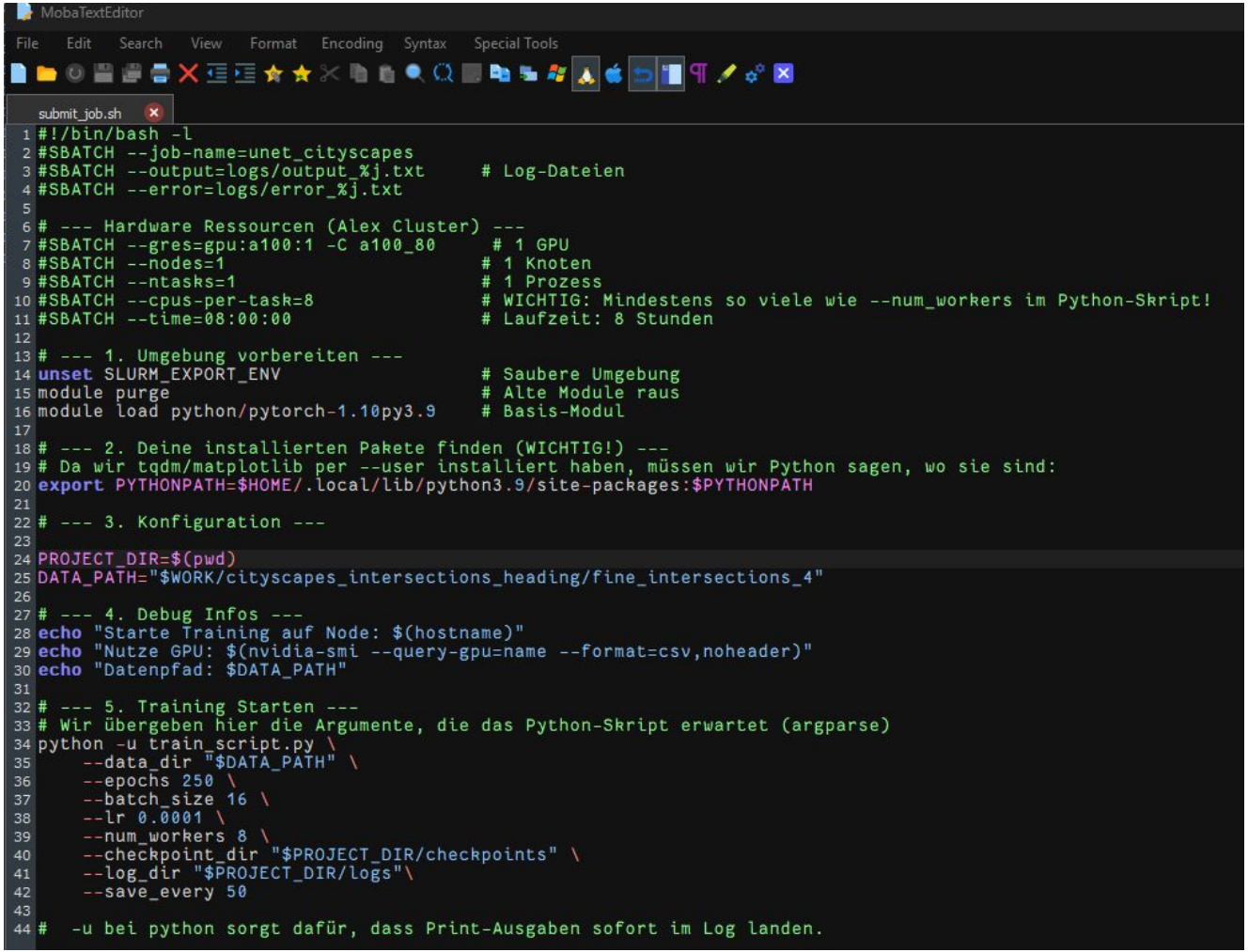


- Zoom in das Bild auf 70% bzw 90% des Bildes
- Vermeidung eines “Sizes of tensors must match” Errors bei zu kleinen Bildern zwischen den Batches

# Alex NHR@FAU

-Slurm script

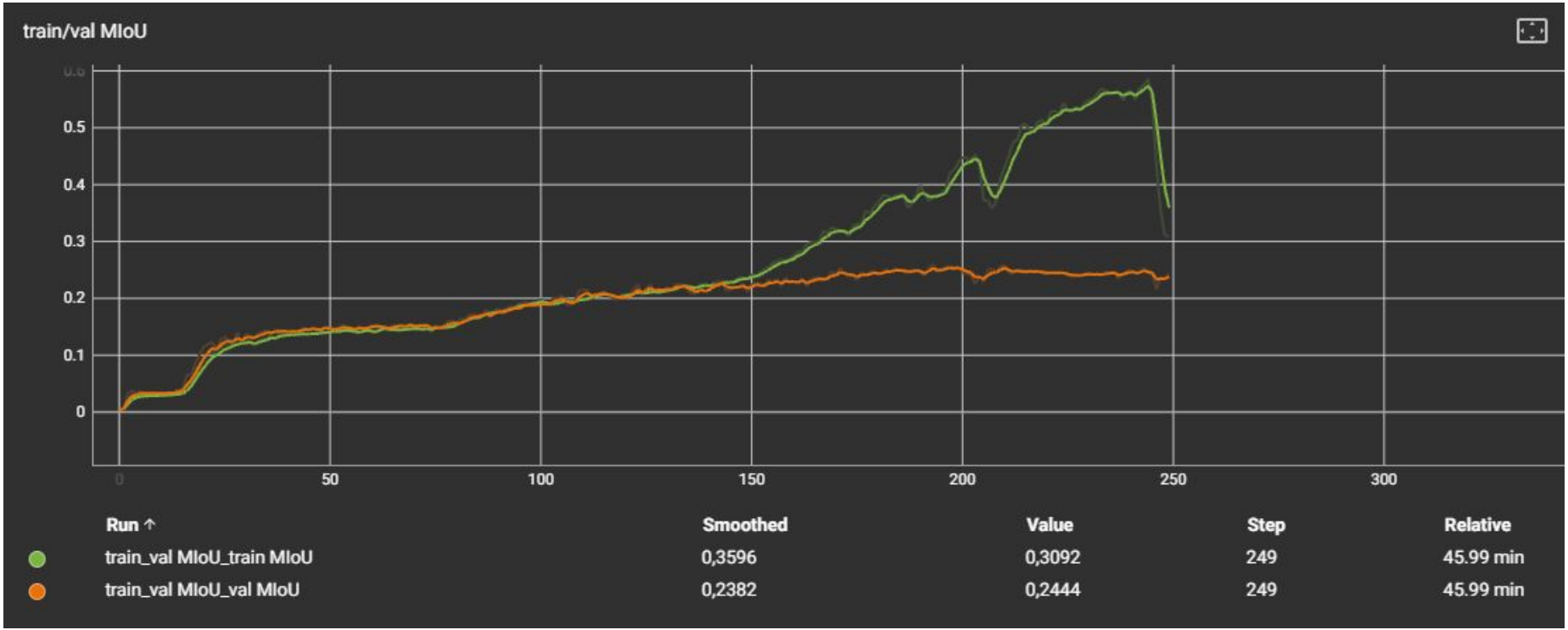
um Job zu starten



# Ergebnisse vom ersten Training mit Tensorboard

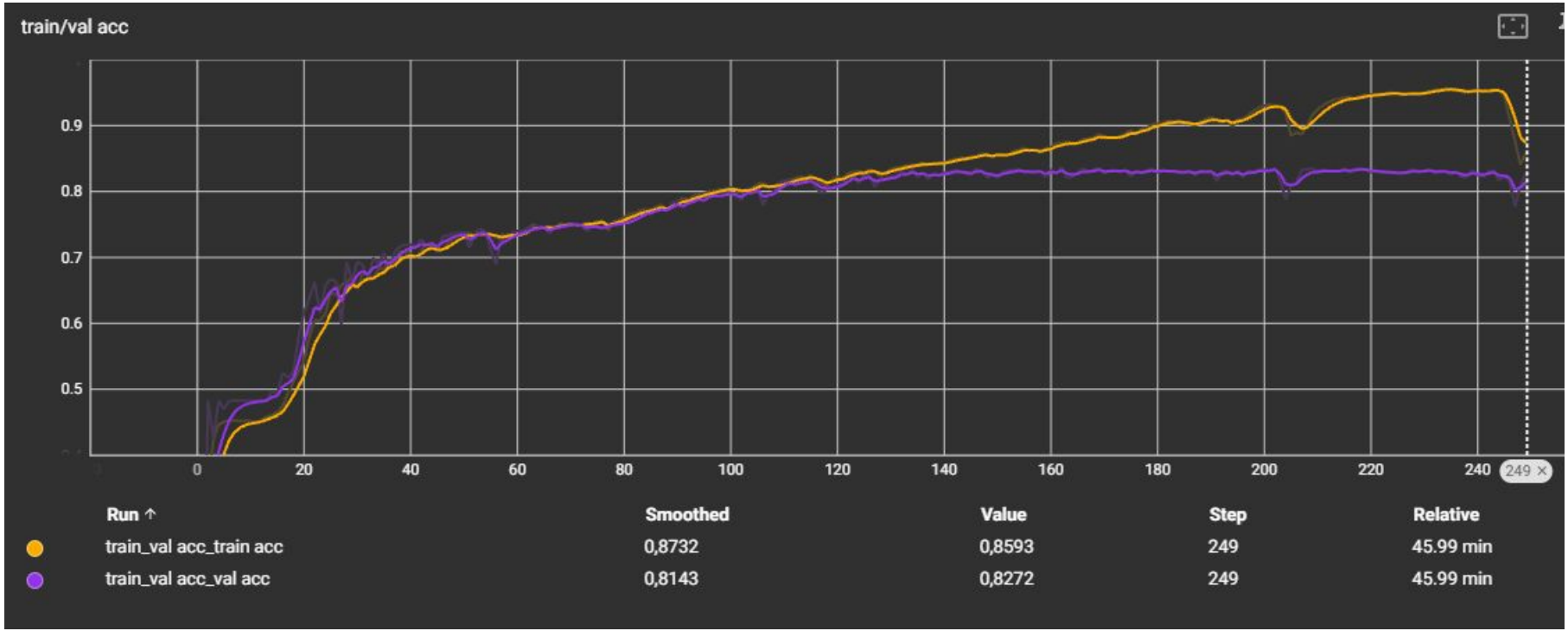
- Modell: U-net
- Datensatz : Fine_4_Heading: 300 Bilder
- 18 Klassen (TrainIDs)
- ignore_index=255

# Ergebnisse vom ersten Training mit Tensorboard




| Run ↑ | Smoothed | Value | Step | Relative |
|---|---|---|---|---|
| train_val MIoU_train MIoU | 0,3596 | 0,3092 | 249 | 45.99 min |
| train_val MIoU_val MIoU | 0,2382 | 0,2444 | 249 | 45.99 min |

# Ergebnisse vom ersten Training mit Tensorboard




| Run ↑ | Smoothed | Value | Step | Relative |
|---|---|---|---|---|
| train_val acc_train acc | 0,8732 | 0,8593 | 249 | 45.99 min |
| train_val acc_val acc | 0,8143 | 0,8272 | 249 | 45.99 min |

# Ergebnisse vom ersten Training mit Tensorboard

